`SlugField` all inherit from `CharField`, adding validation for email addresses, URLs, and slug identifiers (mixes of letters, numbers, and dashes, often used as part of a URL). The `EmailField` defaults to a max length of 254, the `URLField` to 200, and the `SlugField` to 50. Validation for these three variants almost always get done at the `Model` level unless the underlying database has a special storage type for them.

A close relative of the `CharField` is the `TextField`. A `CharField` requires a fixed amount of allocated space, whereas the `TextField` is an open block. This variation is due to an implementation detail at the database level. Some databases implement both of these fields as the same data type, while others treat them separately. In the Django Admin or in web forms, the `CharField` is presented as an entry widget, while the `TextField` is presented as a multiline text widget. This is default behavior and is under your control if you wish. You'll learn more about this in chapter 7, which covers forms and user input.

Django provides a wide variety of numeric fields, but they all boil down to either integer or float storage. The `IntegerField` and `FloatField` are the base mechanisms for storing ints and floats, while variations, such as `BigIntegerField` and `PositiveSmallIntegerField`, change the validation and possibly the amount of storage required by the data. The latter is database dependent.

Chapter 6 covers user data, and in it you will learn about file uploads. These use a field that holds a file path with file management on your server. The general version of this is the `FileField`, while the `ImageField` adds validation that the uploaded content was a recognized image format.

A recent addition to the list of fields is the `JSONField`. Although you could store JSON as text generally, some databases have implemented a datatype that is JSON aware. The advantage of this field is you access the corresponding attribute as if it is a Python object. Under the covers, the field automatically serializes the object into JSON and stores the result as encoded text. There are limitations to which database and database version support this field, and any Python object in the field must be serializable to the JSON format. SQLite is compatible with the `JSONField` if a particular extension is enabled. The extension is called JSON1, and whether it is on by default depends on the SQLite distribution and the operating system you're working on. See http://mng.bz/v8Xa for information on how to determine if you can use this extension in your environment.

4.9 Fixtures

Manually entering data into your system is tedious. Django provides two management commands that let you import and export data. Django calls the serialized data a *fixture*. Fixtures come in a variety of formats, including JSON, JSONL, and XML. You can also use YAML if you have the PyYAML library installed.

A fixture is a text version of the data you have in your database. This includes all the information needed to create the exact state of the tables involved. Object IDs are in the fixture, both as primary keys of an object and to express inter-object relationships. The `dumpdata` management command outputs the fixture data to the screen in